Entrega 1 · Arquitectura Big Data local · Steam Reviews 2021

Big Data & Cloud Computing · Tecnun, Universidad de Navarra · Curso 26-27

Equipo: Manuel Fernández Pérez-Ricarte, Marc Pich-Aguilera Béjar y Nora Cepas Zapico.

Repositorio: https://github.com/Marc456-jpg/BigDataTeam (rama `main`). Todas las cifras de este documento salen de archivos del repositorio, citados entre paréntesis; las evidencias están en `evidence/`.

Resumen

Hemos montado en un solo PC una arquitectura Big Data completa:

- **HDFS** (1 NameNode y 2 DataNodes, replicación 2);
- **Spark 4.2** (master y worker);
- **YARN**, solo para MapReduce;
- **Jupyter**.

Todo arranca con un solo `docker compose up -d`. Un único comando, `python run_pipeline.py`, lleva el dataset **Steam Reviews 2021** (8,17 GB, 21,7 M de reseñas de 315 juegos) de la descarga al modelo de Machine Learning, en 15 pasos y ≈ 20 min:

1. ingesta en zonas de HDFS;
2. ETL a Parquet particionado, con control de calidad y cuarentena;
3. MapReduce frente a RDD;
4. tres consultas y una optimización medida;
5. regresión logística.

Caso de uso. Un *publisher* de videojuegos quiere vigilar la opinión sobre sus juegos: qué juegos valora mejor la comunidad, cómo cambia la opinión con las horas jugadas, qué reseñas consideran útiles los jugadores de cada idioma y si se puede predecir que una reseña será positiva.

Medida clave	Valor
Filas limpias en `/curada/reviews`	21.612.437 (7 corruptas a cuarentena, 134.927 duplicadas quitadas)
Parquet frente a CSV	53,7 % del tamaño; consulta ≈ 50 veces más rápida (0,9 s frente a 44,8 s, mediana de 3 medidas)
RDD frente a MapReduce	13,3 s frente a 61,5 s, con resultado idéntico
Optimización de Q1	4,6 s → 1,1 s (4,3 veces)
Modelo de ML	AUC 0,7451 con 21,6 M de reseñas
Pipeline completo (pasos 4-15)	19,9 min (≈ 21 min desde que el paso 7 repite cada medida 3 veces)

A1. El problema y las 5V

El *publisher* necesita dos cosas:

- **una vista histórica fiable**, recalculable sobre todas las reseñas, para comparar juegos, idiomas y años;
- **en el futuro, alertas rápidas**, por ejemplo cuando un juego sufre un *review bombing*: una avalancha coordinada de reseñas negativas.

Esta entrega construye la primera; la segunda se deja preparada en el diseño (A2).

V	¿Aplica?	En nuestro dataset (medido)	Qué decisión provoca
Volumen	**Sí**	Un CSV de 8,17 GB con 21.747.371 reseñas de 315 juegos; en HDFS ocupa 16,3 GB con 2 copias. No cabe con holgura en los 7,57 GiB de RAM de Docker	HDFS con bloques de 128 MB y Spark en paralelo; Parquet para leer menos (apartado B)

V	¿Aplica?	En nuestro dataset (medido)	Qué decisión provoca
Velocidad	**No, en el dataset**	Es una foto fija: se publicó en Kaggle en enero de 2021 y no se actualiza. Procesarlo entero tarda ≈ 20 min. Solo aplicaría en el caso de uso, con reseñas nuevas cada día	Procesamiento por lotes; la capa rápida queda como trabajo futuro (A2)
Variedad	**Parcial**	Columnas tabulares (23), más texto libre en al menos 28 idiomas (los que aparecen en 2020, Q3) con saltos de línea y comillas dentro, más un catálogo en JSON de la API de Steam. No hay imágenes, audio ni muchas fuentes distintas	Esquema explícito y `multiLine` en el ETL; el JSON crudo se guarda tal cual en `/raw`
Veracidad	**Sí**	7 filas corruptas, 134.927 `review_id` duplicadas (0,62 %), 545 votos con el valor imposible 4.294.967.295 ($2^{32}-1$), 33.742 reseñas sin texto y 25.682 sin horas (`evidence/resultados/calidad_etl_reviews.json`). El precio del catálogo es el de 2026, no el de cada reseña	Cuarentena, deduplicación, avisos y verificación independiente (E); el ML no usa el precio
Valor	**Sí**	El 87,4 % de las reseñas son positivas: el valor está en distinguir juegos, idiomas y momentos (Q1-Q3) y en anticipar la opinión	Las 3 consultas (D) y el modelo (E)

Conclusión: nuestro problema es sobre todo de **Volumen** y **Veracidad**. La **Velocidad no aplica** al dataset, porque es un histórico cerrado; la tendría el caso de uso real, y por eso la arquitectura la deja prevista (A2).

A2. Data Lake y arquitectura Lambda

Caso de uso (hipotético, definido por el equipo). Un *publisher* con decenas de juegos en Steam quiere:

1. un *dashboard* actualizado cada día con la valoración de sus juegos por idioma y por año;
2. consultas *ad hoc* de sus analistas sobre todo el histórico (por ejemplo, ¿qué juegos mejor valorados tienen más de 1.000 reseñas?);
3. un modelo que estime si una reseña nueva será positiva;
4. más adelante, avisos en minutos si un juego sufre un *review bombing*.

De ahí salen las dos decisiones:

Data Lake, no Data Warehouse. Un Data Warehouse exige fijar el esquema antes de cargar y guarda datos ya modelados. Nuestro dataset mezcla columnas, texto libre multilínea y JSON, y todavía no sabemos todas las preguntas que se harán. Por eso guardamos el original intacto y aplicamos el esquema al leer (*schema-on-read*), en zonas:

Zona HDFS	Contenido	Regla
`/raw`	CSV original y JSON crudo del catálogo	Nunca se modifica: permite rehacer todo
`/curada`	Parquet limpio particionado por `year`, catálogo limpio y entrada del MapReduce	Se reescribe entera en cada ETL (idempotente)
`/resultados`	Consultas, comparativas, informe de calidad y modelo	Pequeño; se copia a `evidence/resultados`
`/quarantine`	Filas inválidas con su motivo	Se revisan, no se pierden

La zona `/resultados` podría alimentar después un Data Warehouse o un *dashboard*: el lago no lo impide, lo prepara.

	Data Lake (elegido)	Data Warehouse
A favor	Guarda el CSV y el JSON originales tal cual (se puede rehacer todo); admite texto libre y JSON; esquema al leer, para las consultas *ad hoc* que aún no conocemos	Consultas SQL rápidas y gobernadas sobre datos ya modelados; ideal para el *dashboard* fijo

	Data Lake (elegido)	Data Warehouse
En contra	Sin disciplina se convierte en un «pantano»: lo evitamos con zonas, Parquet y controles de calidad	Hay que fijar el esquema antes de cargar; el texto libre multilínea y el JSON encajan mal; se pierde el dato original

Lambda, no Kappa.

- *Kappa* trata todo como un flujo: el histórico se recalcula reproduciendo el registro de eventos.
- *Lambda* separa una **capa batch**, que recalcula todo sobre el histórico, de una **capa rápida**, que procesa solo lo nuevo, y une ambas en una capa de servicio.

Elegimos Lambda por tres motivos:

1. El valor principal está en el histórico completo: 21,6 M de reseñas y 11 años, que se recalculan por lotes en ≈ 20 min.
2. Nuestro dato de origen es un CSV, no un registro de eventos que se pueda reproducir.
3. Las alertas de *review bombing* solo necesitan rapidez sobre lo nuevo.

	Lambda (elegida)	Kappa
A favor	La capa batch recalcula todo el histórico de forma fiable y barata (Spark sobre Parquet); la capa rápida solo mira lo nuevo; encaja con el *dashboard* diario y las consultas *ad hoc*	Un solo camino (todo es flujo) y un solo código; ideal si los datos llegan como eventos
En contra	Dos caminos que mantener: el mismo cálculo podría escribirse dos veces	Rehacer el histórico obliga a reproducir los 21,7 M de eventos por el flujo, y nuestro origen es un CSV, no un registro de eventos reproducible

En esta entrega está construida la capa batch (HDFS + Spark + MapReduce). La capa rápida (por ejemplo, Spark Structured Streaming sobre las reseñas del día) queda como trabajo futuro.

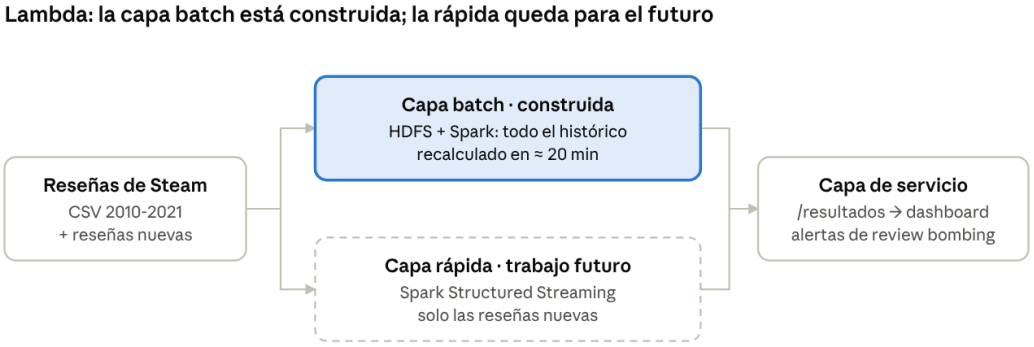


Figura 1. Arquitectura Lambda: capa batch construida, capa rápida como trabajo futuro.

La capa batch es esta entrega; la rápida, en discontinua, es la ampliación prevista para las alertas.

A3. Arquitectura y entorno Docker

Un solo `docker-compose.yml` levanta 6 contenedores con `docker compose up -d`. Otros 2, los de YARN, están en el perfil `yarn` y solo se encienden durante el MapReduce: el propio pipeline los enciende y los apaga.

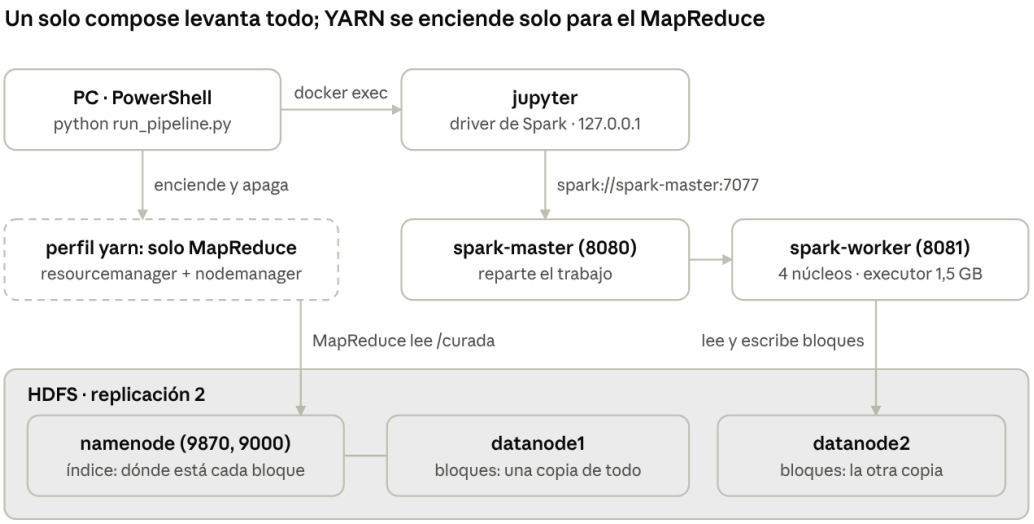


Figura 2. Contenedores del clúster y flujo de datos.

El pipeline lanza cada paso desde el PC; Spark lee y escribe en los DataNodes, y el NameNode solo guarda el índice.

Contenedor	Tope de memoria	Para qué
`namenode`	1 GB (Java 512 MB)	Índice de HDFS
`datanode1`, `datanode2`	768 MB cada uno (Java 384 MB)	Bloques, con replicación 2
`spark-master`	768 MB	Reparte el trabajo de Spark
`spark-worker`	2.816 MB	4 núcleos; un executor de 1,5 GB por programa
`jupyter`	2 GB	Desarrollo y driver de Spark
`resourcemanager` + `nodemanager` (perfil `yarn`)	1 GB + 2.816 MB	MapReduce con Hadoop Streaming (Python 3)

Por qué así, con medidas.

- **Memoria:** Docker tiene 7,57 GiB.
- Con YARN siempre encendido, el pico llegaría a ≈ 7,4 GiB; por eso va en un perfil.
- En el pico de un trabajo de Spark se usan 3,81 GiB (el 50 %).
- **Versiones:** driver, master y worker usan la misma imagen, fijada por su *digest* (Spark 4.2.0, Python 3.13), para evitar `PYTHON_VERSION_MISMATCH`.
- **Seguridad:** Jupyter y la Spark UI solo escuchan en `127.0.0.1`, y las credenciales de Kaggle no se suben a GitHub.

¿Cuántos DataNodes? Dos (tarea 7.4). El profesor pidió justificarlo por eficiencia y según nuestra arquitectura. Lo medimos antes de decidir:

Opción	Disco para `/raw`	Memoria extra	Qué aporta en nuestro caso
1 DataNode	8,2 GB	—	Nada de lo que pide el enunciado: sin bloques repartidos (B) ni tolerancia a fallos (F)
2 DataNodes, replicación 2	16,4 GB	≈ 0,75 GB	Bloques repartidos, una copia de cada bloque en cada nodo y el Desafío 2
3 DataNodes	24,5 GB con replicación 3	≈ 0,75 GB más	Re-replicación visible al caer un nodo, a cambio de copiar 12 GB

Un tercer nodo **no añade tolerancia real**: los tres vivirían en el mismo PC y en el mismo disco físico, y si ese disco falla se pierden todos a la vez. En un despliegue real del caso de uso, con nodos en máquinas distintas, sí se usarían 3 o más, con la replicación 3 por defecto de Hadoop.

B. Almacenamiento en HDFS

Ingesta. `subir_a_hdfs.py` sube el CSV a `/raw` y es idempotente: compara el tamaño exacto y se salta si ya está. `hdfs fsck` muestra el CSV repartido en **61 bloques de 128 MB, cada uno con 2 copias**, una en cada DataNode (`evidence/fase2/2.2_fsck_raw_steam_reviews.txt`).

Lectura del `fsck` (`hdfs fsck /raw/steam_reviews.csv -files -blocks -locations`):

- **8.171.787.229 bytes en 61 bloques.** Son 60 bloques completos de 134.217.728 bytes (128 MB, el tamaño de bloque) y un último bloque de 118.723.549 bytes (≈ 113 MB): 60 × 128 MB + 113 MB = 8,17 GB ✅.
- **`replication=2` y `Live_repl=2` en cada bloque.** Cada bloque lista sus 2 ubicaciones, `172.18.0.3:9866` y `172.18.0.4:9866`, que eran los dos DataNodes ese día. El orden cambia de un bloque a otro: HDFS reparte quién sirve primero cada bloque.
- **`Average block replication: 2.0`, `Missing blocks: 0`, `Corrupt blocks: 0` y `HEALTHY`:** se cumple el factor de replicación configurado, sin pérdidas.
- **Estructura de las zonas:** `evidence/fase2/2.5_hdfs_ls_R_zonas.txt` (`hdfs dfs -ls -R` de `/raw`, `/curada`, `/resultados` y `/quarantine`).

Partición por `year`, decidida midiendo. La inscripción proponía año + mes. El ETL mide el tamaño de cada partición antes de escribir:

- la partición mensual típica (mediana) pesaría **24,7 MB**, y habría 123 archivos pequeños;
- la anual pesa **192,5 MB**: «cientos de MB, no KB», como pide el enunciado.

El resultado son 12 carpetas (`year=2010` … `year=2021`) en Parquet con Snappy: 4.185 MB por copia.

CSV frente a Parquet (`comparar_formatos.py`, misma pregunta: ¿cuántas reseñas recomiendan?):

	CSV (`/raw`)	**Parquet (`/curada`)**	**Parquet, solo 2020**
Tamaño (1 copia)	7.793 MB	4.185 MB (53,7 %)	—
Tiempo	44,8 s (42,8-46,5)	0,9 s (0,8-2,1)	0,4 s (0,4-0,5)

Método de medida. Cada consulta se mide 3 veces, en ronda (CSV, Parquet, Parquet 2020, CSV…), en el mismo clúster y sin otros programas abiertos en el PC. La tabla da la mediana y, entre paréntesis, el rango (`evidence/resultados/comparativa_csv_vs_parquet.json`, ejecución `20261020_011845`). Aprendimos dos cosas al medir: una sola medida engaña, porque la primera lectura de Parquet es la más lenta (2,1 s; probablemente el arranque en frío) y una ejecución anterior con una sola medida dio «15 veces»; y la carga del PC también cuenta: con un vídeo reproduciéndose, el CSV tardó un 28 % más (62,5 s de mediana, ejecución `20261019_233334`). Sin nada más abierto, Parquet es **≈ 50 veces** más rápido.

Parquet gana por tres motivos:

- es columnar: solo lee la columna `recommended`;
- está comprimido;
- con un filtro por año, Spark solo abre esa carpeta (*partition pruning*).

El CSV, además, tiene reseñas con saltos de línea. Leerlo bien exige `multiLine`, y con esa opción Spark no puede trocear el archivo: la primera pasada del ETL la hace una sola tarea (≈ 10 MB/s).

C. MapReduce frente a RDD

El cálculo: el porcentaje de reseñas positivas de cada juego, que es la base de Q1. Se calcula de dos formas sobre la misma entrada: 8 archivos de texto `app_id<TAB>recommended` exportados desde la zona curada. Así la comparación es justa.

- **Hadoop Streaming en YARN:** un *mapper* y un *reducer* en Python 3 (`mapreduce/positivos_por_juego/`). Leen bytes, no texto, para no romperse con el UTF-8 (decisión 37).
- **Spark RDD:** `map` + `reduceByKey` (`positivos_por_juego_rdd.py`).
- **Comparación automática:** el paso 13 del pipeline compara tiempos y resultados (`mapreduce_vs_rdd.json`). Así lo pidió el profesor: nada a mano.

	Hadoop Streaming en YARN	**Spark RDD**
Cálculo	61,5 s	13,3 s
Comando entero (con arranque)	80,1 s	22,6 s

	Hadoop Streaming en YARN	Spark RDD
Resultado	315 juegos	**Idéntico**

Spark es ≈ 4,6 veces más rápido en el cálculo (3,5 contando el arranque). Nota honesta: estos tiempos, y los de la optimización de Q1 y del ML, son de la ejecución 20261019_233334, hecha con un vídeo abierto en el PC. Con el PC libre (19 oct, 18:39) fueron 56,4 s frente a 11,9 s (4,7 veces): la proporción apenas cambia. **Por qué:**

- `reduceByKey` suma dentro de cada trozo antes de enviar nada, como un *combiner*: por la red viajan como mucho 2.520 totales parciales.
- Nuestro MapReduce, sin *combiner*, envía 21,6 M de pares al *shuffle*.
- Además, YARN arranca un contenedor de Java por cada tarea, mientras que Spark reutiliza el executor que ya tiene arrancado.

Añadir `-combiner` al MapReduce acortaría la diferencia; no se ha probado (limitaciones).

Map, shuffle y reduce con nuestros pares clave-valor:

1. **Map.** Cada línea de entrada, por ejemplo `730<TAB>true`, se convierte en el par `("730", 1)` (o `("730", 0)` si no recomienda). Hay 8 tareas map, una por archivo de entrada, que emiten 21.612.437 pares.
2. **Shuffle.** Hadoop reparte los pares por clave (`app_id`), los ordena y los escribe en disco, de modo que el reducer recibe juntos todos los `("730", …)`. Sin *combiner*, viajan los 21,6 M de pares. En Spark, `reduceByKey` suma primero dentro de cada partición: de cada una salen, como mucho, 315 parejas `("730", (reseñas, positivas))`, en total como mucho 2.520.
3. **Reduce.** Para cada juego cuenta las reseñas y suma los 1, por ejemplo `730 → (reseñas, positivas, % positivas)`. El resultado son 315 líneas, idénticas en las dos versiones.

Complejidad del código.

- El MapReduce necesita dos programas: el *mapper* (≈ 13 líneas de lógica) y el *reducer* (≈ 22). El *reducer* tiene que detectar a mano dónde acaba cada juego en la entrada ordenada (la variable `app_actual`). Además hay que lanzar un `hadoop jar` con 8 opciones y encender YARN.
- Con RDDs basta una cadena de 3 operaciones (`flatMap → reduceByKey → map`) y 3 funciones pequeñas, ≈ 15 líneas en un solo archivo. Spark se encarga de agrupar por clave.
- **Por qué MapReduce tarda más:**
- escribe en disco entre map y reduce;
- arranca una JVM nueva por cada tarea en un contenedor de YARN;
- sin *combiner*, mueve 21,6 M de pares por el *shuffle*.

Spark reutiliza un executor ya arrancado y combina en memoria antes de mover nada.

D. Procesamiento con Spark

ETL (`etl_reviews.py`, 9,7 min):

- **Esquema explícito de 23 columnas.** Adivinar los tipos obligaría a leer los 8 GB una vez más, y podría equivocarse.
- **`multiLine` y `escape='"'`**, porque hay reseñas con saltos de línea y comillas dentro.
- **Dos pasadas.** CSV → Parquet de paso → curada: el CSV multilínea solo se lee con una tarea, así que ese coste se paga una vez y el resto va en paralelo.
- **Fechas en UTC**, para que el año (la partición) no dependa del PC.
- **Duplicados por `review_id`:** se queda la versión más reciente. La lista de ids repetidos va en *broadcast* (decisión 55). La primera versión ordenaba las 21,7 M de filas con su texto y mató 4 executors por memoria.
- **Grupos de filas de Parquet de 16 MB**, para que escribir no se quede sin memoria (decisión 44).

La limpieza, paso a paso, con los registros antes y después (`evidence/resultados/calidad_etl_reviews.json`):

Paso	Regla y motivo	Acción	Filas
Lectura	`StructType` con 23 columnas y sus tipos (enteros, booleanos, *timestamps* Unix…), nunca `inferSchema`	—	21.747.371 leídas
Tipos	Los *timestamps* Unix se convierten a fecha en UTC, y de ahí sale `year`	Se transforma	—
Corruptas	Filas que Spark no puede interpretar con el esquema (modo PERMISSIVE)	A `/quarantine`, con su motivo	− 7
Nulos en columnas clave	Sin `review_id`, `app_id`, `recommended` o fecha la fila no sirve para ninguna pregunta	A `/quarantine`	− 0
Fechas fuera de rango	Antes del 12-09-2003 (nacimiento de Steam) o después de 2021	A `/quarantine`	− 0
Duplicados	La misma `review_id` aparece varias veces: se queda la versión editada más reciente (`timestamp_updated`)	Se eliminan	− 134.927
Valores atípicos	545 votos con $2^{32}-1$ (contador roto), 33.742 reseñas sin texto y 25.682 sin horas	Se cuentan y se avisa; no se borran porque el resto de la fila es válido. Q3 filtra los votos imposibles	0
Resultado	Rango de fechas real: del 19-11-2010 al 23-01-2021	—	**21.612.437**

Las tres preguntas (`consultas.py`, de dificultad creciente):

Pregunta	Técnica	Resultado
Q1. Los 10 juegos con mayor % de reseñas positivas (mínimo 1.000), con géneros, desarrollador y precio	Agrupación + *join* con el catálogo	1.º ULTRAKILL (99,53 %), 2.º Senren * Banka (99,40 %), 3.º A Short Hike (99,14 %). Sobre todo indies; ninguno de los más reseñados
Q2. Horas jugadas al reseñar, según recomiende o no	Media y mediana	Quien no recomienda llevaba más horas: mediana 37,3 h frente a 30,8 h. La media es 4,5-5,4 veces la mediana: la disparan unos pocos jugadores con miles de horas
Q3. Top 5 de juegos por idioma según votos de «útil» en 2020	Ventana `DENSE_RANK` por idioma	28 idiomas. En inglés lidera Among Us (171.445 votos); en chino simplificado, Wallpaper Engine. Solo se lee `year=2020`

Optimización medida de Q1 (`optimizar_q1.py`, planes en `evidence/fase5/`):

Versión	***Join***	***Shuffles***	Tiempo
V1 · *join* primero	SortMergeJoin	2	4,6 s
V2 · *join* primero, catálogo en *broadcast*	BroadcastHashJoin	1	2,0 s
V3 · agregar antes + *broadcast*	BroadcastHashJoin	1	1,1 s (4,3 veces más rápida)

Las tres dan el mismo top 10. El catálogo tiene 315 filas: enviarlo entero a cada executor (*broadcast*) evita mezclar 21,6 M de filas. Y agregando antes del *join*, por la red solo cruzan ≈ 7.160 totales parciales (`rowCount=7.16E+3` en el plan).

Cómo se lee el plan. En `explain("formatted")`, cada **`Exchange`** es un *shuffle*: Spark reparte filas por la red, y la Spark UI lo muestra como un límite entre dos **etapas** (*stages*).

- **V1** tiene 2 `Exchange`:
- uno para juntar las reseñas por `app_id` antes del `SortMergeJoin`;
- otro para el catálogo.
- **V2 y V3** cambian el segundo por un **`BroadcastExchange`**, que no es un *shuffle*: el catálogo, de 315 filas, se copia entero a cada executor.
- En V3, además, el `HashAggregate` parcial va **antes** del único `Exchange`, así que por la red cruzan totales parciales y no filas.

Las capturas de la Spark UI (`evidence/fase5/`) muestran estas etapas: el DAG de V1, con el `SortMergeJoin`, y el de V3, con el `BroadcastHashJoin`, más la pestaña Stages (5.2_07_spark_ui_stages_a/b), que mide cuánto cruza la red en cada versión. En V1, una etapa de 34 tareas escribe 20,8 MiB de shuffle, y la etapa del join (5 tareas) los lee. En V2 el shuffle baja a 379,8 KiB, y en V3 a 188,9 KiB: 56 y 113 veces menos datos por la red. Cada lectura del Parquet ocupa solo 8,4 MiB de entrada, porque Spark lee únicamente las columnas que usa la consulta. Las 9 etapas «skipped» son shuffles ya calculados que Spark reutiliza al repetir la medida.

Por qué esta optimización: el *join* une 21,6 M de filas con una tabla diminuta. Es el caso de libro del *broadcast join*, y agregar antes reduce lo que se mueve. Cada versión se mide 2 veces y se toma el mejor tiempo, y las tres dan el mismo top 10. Los tiempos de la tabla se tomaron con un vídeo abierto en el PC; con el PC libre (19 oct, 18:39) fueron 4,0 → 1,6 → 0,9 s (4,6 veces).

E. Integración: un pipeline reproducible

Punto de entrada único: `python run_pipeline.py`, que se lanza en el PC y ejecuta 15 pasos con `docker exec`. Está fuera de los contenedores porque tiene que manejar a la vez Spark, Hadoop y el encendido de YARN. El profesor pidió que el MapReduce y Spark fueran en el mismo pipeline, con la comparación automática.

- Cada ejecución guarda su registro y su resumen con los tiempos (`evidence/ejecuciones/`).
- Se detiene en el primer paso que falla.
- Admite `--desde`, `--hasta` y `--solo`.

Configuración separada del código. Ningún script lleva rutas ni parámetros escritos dentro: todo sale de `config/pipeline.yaml`. Incluye las zonas de HDFS, las memorias, la zona horaria, el umbral de partición, el rango de fechas válido y los parámetros de las consultas y del ML.

Idempotencia.

- Las descargas y la subida se saltan si ya están hechas.
- Todas las escrituras de Spark reemplazan lo anterior.
- El MapReduce borra su salida antes de empezar.

Comprobado el 19 oct, rehaciendo todo y comparando con el commit anterior:

- Q1-Q3, el MapReduce y el plan V1 salieron **idénticos byte a byte**;
- el RDD dio las mismas 315 líneas en otro orden, porque `reduceByKey` no ordena.

Se repitió la noche del 19 al 20 oct con el código final, ya con todo el entorno en el YAML (ejecuciones `20261019_233334` y `20261020_011845`). Según `git diff --stat`, solo cambiaron los 3 JSON que guardan tiempos; Q1-Q3, el MapReduce, el RDD, el informe de calidad y los 3 planes de `evidence/fase5/` salieron idénticos byte a byte.

Calidad de datos.

```
21.747.371 filas leídas del CSV
−        7 corruptas                    → /quarantine/reviews (con su motivo)
−  134.927 duplicadas por review_id    (se queda la más reciente)
= 21.612.437 filas en /curada/reviews
```

- **Se cuentan pero no se borran:** 33.742 reseñas sin texto, 25.682 sin horas y 545 votos con el valor imposible $2^{32}-1$.
- **Verificación independiente:** `verificar_curada.py` vuelve a leer lo escrito y comprueba 4 reglas: filas = informe, sin duplicados, sin claves nulas y cada fila en la carpeta de su año.
- **Cuarentena:** al profesor le gustó la idea de guardar las filas inválidas en vez de solo contarlas.

Control	Dónde	Acción definida
Filas corruptas, nulos en columnas clave, fechas fuera de rango	ETL	**Registrar:** la fila va a `/quarantine` con su motivo (exportada a `evidence/resultados/cuarentena_reviews.json`)
Duplicados de `review_id`	ETL	**Corregir:** se queda la versión más reciente

Control	Dónde	Acción definida
Valores imposibles ($2^{32}-1$), reseñas sin texto o sin horas, horas negativas, `weighted_vote_score` fuera de [0, 1]	ETL	**Avisar:** se cuentan en el informe de calidad
Filas escritas = informe, sin duplicados, sin claves nulas, cada fila en la carpeta de su año	`verificar_curada.py` (paso 5)	**Detener:** si algo no cuadra, devuelve 1 y el pipeline se para en ese paso

Línea base de MLlib. Una `LogisticRegression` predice `recommended` con las 21,6 M de reseñas (80 % entrenamiento y 20 % prueba, semilla 42; 226 s con el PC libre y 281 s con un vídeo abierto). **Solo usa información que existía al escribir la reseña**, para no filtrar el futuro:

Variables (23)	Por qué tienen sentido
Horas jugadas al reseñar (log) y si no constan	Quien lleva más horas ya sabe si le gusta el juego (Q2)
Longitud del texto (log)	Las quejas suelen ser más largas
Compra en Steam, recibido gratis, acceso anticipado	El contexto de la reseña
10 idiomas	Cada comunidad valora de forma distinta
7 géneros del catálogo	Los géneros casi no cambian con el tiempo

Se excluyen:

- las horas totales, los votos y los comentarios, porque ocurren después de la reseña;
- el precio del catálogo, porque es el de 2026 y no el de la fecha de cada reseña (decisión 51, aprobada por el profesor).

Resultado: AUC ROC 0,7451 (0,5 = azar) y AUC PR 0,9469. Se mide con AUC porque el 87,4 % de las reseñas son positivas: con el umbral 0,5 acierta el 87,41 %, igual que decir siempre «sí» (87,43 %). El AUC muestra que sí ordena bien las reseñas.

Lo que más pesa:

- la longitud del texto (−0,50);
- el género Indie (+0,48);
- el chino simplificado (−0,39).

Son correlaciones, no causas.

F. Desafío 2: tolerancia a fallos

Qué se prueba. Que un fallo en mitad de un trabajo no cambia el resultado. El trabajo de prueba es `verificar_curada` (`run_pipeline.py --solo 5`):

- lee las 21,6 M de filas;
- no escribe nada;
- termina con «TODO CUADRA», lo que demuestra que los datos leídos son correctos.

Escenario 1: se apaga un DataNode durante el trabajo (19 oct; `evidence/fase7/7.1_00` a `7.1_16`).

Hora (UTC)	Qué pasó
18:13:10	Arranca el trabajo. A los 2 s, `docker compose stop datanode2`
18:13:33-18:14:58	Cada lectura dirigida a datanode2 falla (`NoRouteToHostException`) y el cliente de HDFS lee la otra copia. Hay 150 avisos y ningún error
18:14:58	**TODO CUADRA** con las mismas 21.612.437 filas, en 112,5 s en vez de ≈ 37 s
18:24:07	El NameNode da datanode2 por muerto: `fsck` HEALTHY, **178 de 178 bloques sub-replicados y 0 perdidos**. No copia nada, porque no hay otro nodo donde poner la segunda copia
18:37	`docker compose start datanode2`. En menos de 1 min hay 2 nodos vivos y 0 sub-replicados, **sin copiar nada**: al volver, el nodo presenta su lista de bloques

Hallazgo: la detección tardó 655 s, no los ≈ 14-20 s previstos.

- El ajuste `dfs.namenode.heartbeat.recheck-interval=2000` del compose, que viene igual del compose del profesor, **no se aplica**: las imágenes bde2020 exigen escribir el guion como `___`.
- `hdfs getconf` devuelve 300.000 ms, el valor por defecto. Con él, un nodo se da por muerto tras 2 × 300 s + 10 × 1 s = 610 s, más la espera hasta la siguiente revisión.
- Lo descubrimos porque teníamos una predicción concreta y la medimos.
- Decidimos no recrear el NameNode la víspera de la entrega (decisión 65). Lo documentamos en el compose y en el README, con la forma de corregirlo.

Escenario 2: se mata el executor de Spark durante el trabajo (19 oct; `evidence/fase7/7.2_00` a `7.2_05`). Un comando espera a que aparezca el executor, deja pasar 8 s y lo mata con `pkill`.

Hora (UTC)	Qué pasó
20:33:42	Arranca el executor 0
20:33:50	Se mata. El driver anota `Lost executor 0 … code 143` y pierde las 4 tareas en marcha (8.0-11.0 de la etapa 3.0)
20:33:50 →	Spark lanza el executor 1, que repite **las 12 tareas que el 0 había hecho en la etapa 3.0**: las 4 a medias y **las 8 ya terminadas**
Final	**TODO CUADRA** con las mismas cifras, en 30,2 s

Las 8 tareas ya terminadas se repiten porque su resultado intermedio, el que la etapa deja preparado para el *shuffle*, estaba en el executor muerto. Spark no guarda copias, sino la «receta» (el **linaje**): vuelve a leer esas partes del Parquet en HDFS y rehace el cálculo.

Evidencia previa, no provocada. Spark ya se había recuperado solo de executors perdidos por memoria:

- 4 en el ETL del 18 oct;
- 2 en el ML;
- 2 en la pasada 1 del ETL del 19 oct, que salió bien al tercer intento de sus tareas.

En los tres casos Spark repitió solo las tareas perdidas y el trabajo siguió; el del 18 oct se verificó después con las 4 reglas de calidad. También vimos su límite: ese mismo día, con executors de 1 GB, una etapa falló 4 veces y el ETL abortó. Por eso el executor pasó a 1,5 GB (decisión 61).

Por qué basta con 2 DataNodes (A3): con 3 se vería HDFS copiando 12 GB a un nodo nuevo, pero en el mismo disco físico, así que esa «tolerancia» no existe en un solo PC. Lo esencial se demuestra igual: no se pierde ningún dato, el trabajo termina bien, HDFS sabe qué bloques están en riesgo y se recupera solo.

Lo que nos enseñó el dataset

Conclusiones del análisis de las 21,6 M de reseñas limpias. Cada una dice de dónde sale y separa lo **medido** de lo que es solo una **hipótesis** (una explicación posible que no hemos comprobado).

#	Reflexión	Dato (fuente)
1	**Los veteranos son los más críticos.** Quien no recomienda llevaba más horas al reseñar. Hipótesis: jugadores de muchas horas descontentos con cambios o monetización	Mediana 37,3 h frente a 30,8 h (Q2)
2	**Popular no es lo mismo que bien valorado.** El juego más reseñado es de los peor valorados; los mejor valorados son indies con muchas menos reseñas	PUBG: 1.644.255 reseñas (7,6 % del total), 53,9 % positivas; ULTRAKILL: 5.584, 99,5 % (`mapreduce_positivos_por_juego.tsv`, Q1)
3	**Steam es muy positivo**, así que las clases están desequilibradas: un modelo que dijera siempre «sí» acertaría el 87 %. Por eso el ML se mide con AUC	87,4 % positivas (`ml_regresion_logistica.json`)
4	**2020 fue el año récord**, con más de un tercio de todas las reseñas. Que se deba a los confinamientos es hipótesis: solo coincide en el tiempo	2010: 2.373; 2020: 7.818.161, el 36 % (paso 5 del pipeline)

#	Reflexión	Dato (fuente)
5	**Las reseñas largas suelen ser negativas**: es la variable que más pesa en el modelo. Hipótesis: quien se queja explica más sus motivos	Coeficiente −0,50; Indie +0,48; chino simplificado −0,39 (ML)
6	**El dataset no es una muestra al azar de Steam**, sino de 315 juegos de mucho éxito (sesgo de selección): las conclusiones valen para juegos populares	Los 315 siguen a la venta y solo 11 son gratuitos (catálogo, paso 6)
7	**Hay errores en el origen que ganarían cualquier ranking** si no se limpian: votos con $2^{32}-1$, el valor de un contador que «da la vuelta» al restar 1 a 0 (explicación habitual, no comprobada en Steam)	545 votos imposibles (`calidad_etl_reviews.json`); Q3 los filtra
8	**Una foto, no un flujo:** la última reseña es del 23-01-2021, así que 2021 no se puede comparar con años completos; por eso Q3 usa 2020	Informe de calidad

Cambios respecto a la inscripción

El profesor aceptó cambios siempre que se justifiquen. Se mantienen el dataset, la tabla auxiliar de la API de Steam, la clave `app_id` y la variable objetivo `recommended`. Cambia esto:

Inscrito	Ahora	Motivo
Predictores con `playtime_forever` y votos	Solo variables anteriores a la reseña	Esas columnas ocurren después: el modelo «haría trampa» (lo piden las bases)
Precio del catálogo en el ML	Sin precio; géneros sí	El precio es el de 2026, no el de la reseña. El profesor lo aprobó
Partición año + mes	`year`	Medido: 24,7 MB por mes (archivos pequeños) frente a 192,5 MB por año
Q1 sin la tabla auxiliar	Q1 con *join* al catálogo	Ninguna pregunta usaba el *join* que exigen las bases, y permite medir el *broadcast*
Q2 con la media	Media y mediana	Las horas tienen extremos de miles de horas que disparan la media
Q3 sobre 2021	Q3 sobre 2020	2021 solo tiene 3 semanas de reseñas, con pocos votos acumulados
—	Zona `/quarantine`	Calidad de datos: las filas inválidas se pueden revisar. Idea aprobada por el profesor

Registro de decisiones de diseño

Las más relevantes de las 69 que tomamos. Cada una se midió o se probó antes de decidir.

#	Decisión	Alternativas descartadas	Motivo
1	Data Lake en zonas `raw` / `curada` / `resultados` / `quarantine`	Data Warehouse	Guardar el dato original; admite texto libre y JSON; esquema al leer (A2)
2	Lambda, con la capa batch construida	Kappa	Histórico cerrado y origen en CSV, no en eventos reproducibles (A2)
3	2 DataNodes, replicación 2	1 DataNode; 3 DataNodes	B y F exigen más de uno; un tercero en el mismo disco no añade tolerancia y obliga a copiar 12 GB (A3)
4	YARN en un perfil de Docker, solo durante el MapReduce	YARN siempre encendido	Con YARN encendido la memoria llegaría a ≈ 7,4 de 7,57 GiB
5	La misma imagen de Spark, fijada por *digest*, en driver, master y worker	Imágenes distintas	Evita `PYTHON_VERSION_MISMATCH` y cambios de versión entre PCs

#	Decisión	Alternativas descartadas	Motivo
6	Partición por `year`	Año + mes (lo inscrito)	Medido: 24,7 MB de mediana por mes (archivos pequeños) frente a 192,5 MB por año
7	ETL en dos pasadas: CSV → Parquet de paso → curada	Una sola pasada	El CSV multilínea solo se lee con una tarea; ese coste se paga una vez
8	Duplicados: lista de ids repetidos en *broadcast*	Ordenar las 21,7 M de filas con una ventana	La versión con ventana mató 4 executors por memoria
9	Zona de cuarentena	Descartar y solo contar	Las filas inválidas se pueden revisar; el profesor la aprobó
10	Executor de 1.536 MB	1 GB	Con 1 GB y 4 tareas a la vez, el ETL perdió 6 executors y abortó
11	La entrada del MapReduce se exporta desde la zona curada	Leer el CSV crudo	Hadoop Streaming lee línea a línea y el CSV tiene saltos de línea; misma entrada = comparación justa
12	Q1 con *broadcast* y agregando antes	*Sort-merge join*	4,3 veces más rápida, con el mismo resultado (D)
13	ML solo con variables previas a la reseña, sin precio, medido con AUC	Horas totales y votos; precio; acierto	Evitar la fuga de datos; el precio es de 2026; 87,4 % de positivas
14	Orquestador en Python en el PC (`run_pipeline.py`)	Notebooks en secuencia; script dentro de un contenedor	Spark, Hadoop y YARN en el mismo pipeline, con la comparación automática que pidió el profesor
15	Todas las rutas y parámetros, también los del entorno, en `config/pipeline.yaml`	Rutas en el código	Cambiar el entorno sin tocar código (E)
16	No corregir el `recheck-interval` la víspera de la entrega	Corregirlo y recrear el NameNode	Riesgo y tiempo; queda documentado, con la forma de corregirlo (F)
17	Medir las comparaciones de tiempo varias veces (mediana) y sin otros programas en el PC	Una sola medida; medir con el PC en uso	Una medida única dio «15 veces» y, con un vídeo abierto, el CSV tardó un 28 % más; con la mediana y el PC libre, ≈ 50 veces (B)

Problemas, limitaciones y lecciones

Problema real	Cómo se resolvió
El ETL se quedaba sin memoria con executors de 1 GB: 6 executors perdidos y la etapa abortada el 19 oct	Executor de 1,5 GB (el worker ya ofrecía 2 GB y solo se usaba 1), grupos de Parquet de 16 MB y ML en trozos de 32 MB. Resultado: 0 executors perdidos y ETL en 9,7 min
El disco virtual de Docker crece ≈ 25-38 GB con cada ETL y no encoge: C: bajó a 3,81 GB libres	Compactación con `fstrim` + `diskpart` (de 114,2 a 35,5 GB, sin perder datos); procedimiento en el README
Quitar duplicados ordenando 21,7 M de filas con su texto mató 4 executors	Lista de ids repetidos en *broadcast*
Java 8 no respeta el tope de memoria del contenedor	`HADOOP_HEAPSIZE_MAX` en cada servicio de Hadoop
La detección rápida de DataNodes caídos no funcionaba	Descubierto en el Desafío 2 y documentado (apartado F)

Limitaciones.

- **Clúster simulado en un solo PC.** No hay localidad de datos, y la replicación no protege de un fallo del disco físico.
- **Pocas repeticiones.** CSV frente a Parquet se mide 3 veces (mediana) y cada versión de Q1, 2 veces (la mejor). El resto (MapReduce, RDD, ML y los pasos del pipeline) son de una ejecución;

entre dos ejecuciones variaron ≈ 10 % (MapReduce: 56,4 s y 61,5 s, esta última con un vídeo abierto en el PC), aunque las proporciones entre versiones apenas cambian.
- **El MapReduce no tiene *combiner*.**
- **El modelo es una línea base sin el texto** (NLP), que con ≈ 8 GB de RAM sería muy costoso.
- **El catálogo es el de hoy**, no el de la fecha de cada reseña.
- **La detección de nodos caídos tarda ≈ 10-15 min.**

Lecciones.

1. **Medir antes de decidir.** La partición, el número de DataNodes y la memoria del executor se decidieron con números, no por intuición.
2. **Predecir y comparar.** Anotar qué esperábamos antes de cada prueba destapó errores que habrían pasado desapercibidos, como el ajuste de 14-20 s que nunca se aplicaba.
3. **En Big Data, la memoria y el disco mandan tanto como la CPU.** Los fallos más caros fueron por memoria del executor y por el disco del PC, no por código.

Anexo: dónde está cada evidencia

Todo está en el repositorio, en `evidence/`.

Apartado	Evidencia
A · Entorno	`fase1/`: apagado limpio, YARN, Jupyter, prueba de humo, WordCount, Streaming con UTF-8 y arranque desde cero con un solo comando
B · HDFS	`fase2/2.2_fsck_raw_steam_reviews.txt` (61 bloques × 2 copias); fase2/2.5_hdfs_ls_R_zonas.txt (estructura de las zonas); `resultados/comparativa_csv_vs_parquet.json`
C · MapReduce y RDD	`fase4/`; `resultados/comparativa_mapreduce_vs_rdd.json`
D · Spark	`fase3/` (ETL y verificación); `fase5/5.2_plan_*.txt` (planes de `explain`) y capturas de la Spark UI; `resultados/q1…q3`
E · Integración	`ejecuciones/20261019_183937/` (ejecución final: registro y tiempos) y 20261019_233334/ (pasos 5-15 con el código final) y 20261020_011845/ (paso 7: 3 medidas sin otros programas en el PC); resultados/cuarentena_reviews.json (las 7 filas en cuarentena); `resultados/calidad*`; `fase6/` (ML); `resultados/ml*`
F · Desafío 2	`fase7/7.0_*` (medidas para decidir), `7.1_00-7.1_16` (escenario 1) y `7.2_00-7.2_05` (escenario 2)
Ejecución final y disco	`fase8/8.1_01-8.1_11`